Task Discovery Service (TDS): A Dual-Architecture Service Registry for Distributed Systems

Final Year Project Report

February 2026

1 Introduction and Motivation

In large distributed systems, one weak point can force the whole platform to slow down: static service routing. The core issue in the system I worked on at Cubic Transportation Systems was not input handling, but rigid, hard-coded output endpoints. Although the component accepted requests from a dynamic number of devices, every routing change required redevelopment and full regression testing. This made scaling expensive and slow, and over time the original resolvers had to be replaced with load balancers as demand outgrew the fixed design. This project began from that operational bottleneck and asks how service discovery can be made both dynamic and scalable.

1.1 Problem Statement

The problem lay in two key aspects

1. The component was not dynamic even though it performed the same function for any given reuqest from the devices beneath it.

2. The server was mainly overloaded becasue it accepted the data from the requesting device and passed it on to the resolver. The data was never operated upon or even observed by the component.

I proposed to address these issues by designing an architecture that would perform and scale with the system.

1.2 Research Objectives

This project addresses the service discovery problem by developing the Task Discovery Service (TDS), a flexible registry system that supports both centralized and peer-to-peer (P2P) operational modes within a unified architecture. The primary objectives are:

1. Design and implement a dual-mode service registry supporting both centralized and P2P architectures

2. Develop efficient algorithms for service registration, query routing, and load balancing

3. Create a comprehensive monitoring interface for real-time system visualization
4. Evaluate performance characteristics under various network conditions and load patterns
5. Provide multiple transport and security options including UDP, TCP, and mutual TLS authentication

1.3 Contribution

The main contribution of this work is a production-ready service discovery system that eliminates the operational mode dichotomy present in existing solutions. TDS allows development teams to begin with a simple centralized deployment and seamlessly transition to distributed P2P operation as scale requirements increase, without changing client code or protocol definitions. This architectural flexibility, combined with comprehensive security features and real-time monitoring capabilities, represents a significant advancement in practical service discovery systems.

2 Project Background and Context

2.1 Service Discovery in Distributed Systems

Service discovery is a fundamental component of distributed systems architecture, enabling dynamic binding between service consumers and providers. The challenge arises from the ephemeral nature of modern cloud-native applications, where service instances may spawn and terminate rapidly in response to load, failures, or deployment activities.

Traditional approaches to this problem fall into several categories:

- **DNS-based discovery**: Simple but lacks real-time updates and health checking
- **Configuration files**: Static and requires redeployment for changes
- **Centralized registries**: Provides consistency but creates infrastructure dependencies
- **Distributed hash tables**: Offers scalability but introduces complexity

2.2 Related Work

Several prominent systems address service discovery:

etcd is a distributed key-value store using the Raft consensus algorithm. It provides strong consistency guarantees and is widely used in Kubernetes for service coordination. However, it requires a cluster of at least three nodes for production deployments and adds operational complexity.

Consul by HashiCorp offers comprehensive service mesh capabilities including health checking, key-value storage, and multi-datacenter support. While feature-rich, its complexity makes it overkill for smaller deployments.

ZooKeeper provides distributed coordination with strong consistency guarantees but has a reputation for operational difficulty and lacks modern cloud-native features.

Chord DHT pioneered structured peer-to-peer systems using consistent hashing to distribute data across nodes. Its logarithmic lookup complexity ($O(\log N)$) established the theoretical foundation for scalable distributed systems.

These systems excel in their respective domains but require commitment to a specific architectural paradigm. TDS differentiates itself by supporting multiple operational modes within a single codebase, allowing practitioners to select the appropriate architecture for their deployment context.

3 Design

3.1 services, tasks, and the client proxy

The diagram below gives an overview of a client machine in this system. With multiple services registering and querying services to the client proxy which is the local endpoint of this solution.

In this architecture, services running on a machine will query their local instance of the client proxy. As far as the services are concerned, that is all that needs to be done for them to advertise their service along with sending periodic heartbeats. The client proxy will also handle any requests these services have for where to send their data once they

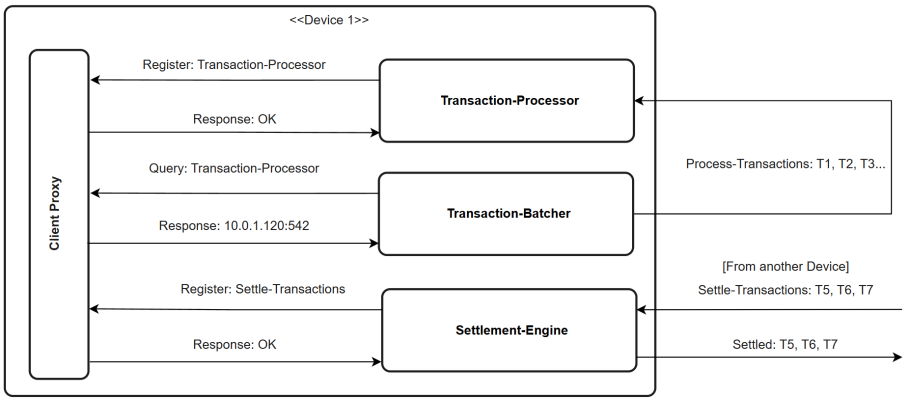

Figure 1: Client machine service registration and discovery via client proxy

have finished the scope of their operations on it. The client proxy is the interface behind which all the mechanisms of this project lie.

3.1.1 Message format

All communication between local services (clients) and the client proxy uses JSON messages in both centralised and P2P modes. The two shared commands are `REGISTER` and `QUERY`.

JSON requests (service → client proxy)

```
{"cmd":"REGISTER","task":"payment","address":"10.0.1.5:8080","
   capacity":10}
{"cmd":"QUERY","task":"payment"}
```

JSON response format

```
{"status":"OK"} //Register - OK
{"status":"OK","address":"10.0.1.5:8080"} //Successful query
{"status":"NOTFOUND"} // queried task not found
{"status":"ERR","error":"task required"} //example error: no task
    in query request
```

These message types are intentionally minimal so the same client-side behavior works regardless of whether the proxy is currently forwarding to a central server or to the P2P registry.

3.2 Centralised mode

We now move outside the scope of the single machine registering and querying services to a distributed system containing many such machines. The below figure demonstrates the role the server plays in centralised mode.

3.2.1 load balancing

The server load balances services using a round-robin mechanism for all services under a task. This ensures even distribution of traffic to each service. Configurable on top of this is a weighted round robin mechanism using each registered service's capacity

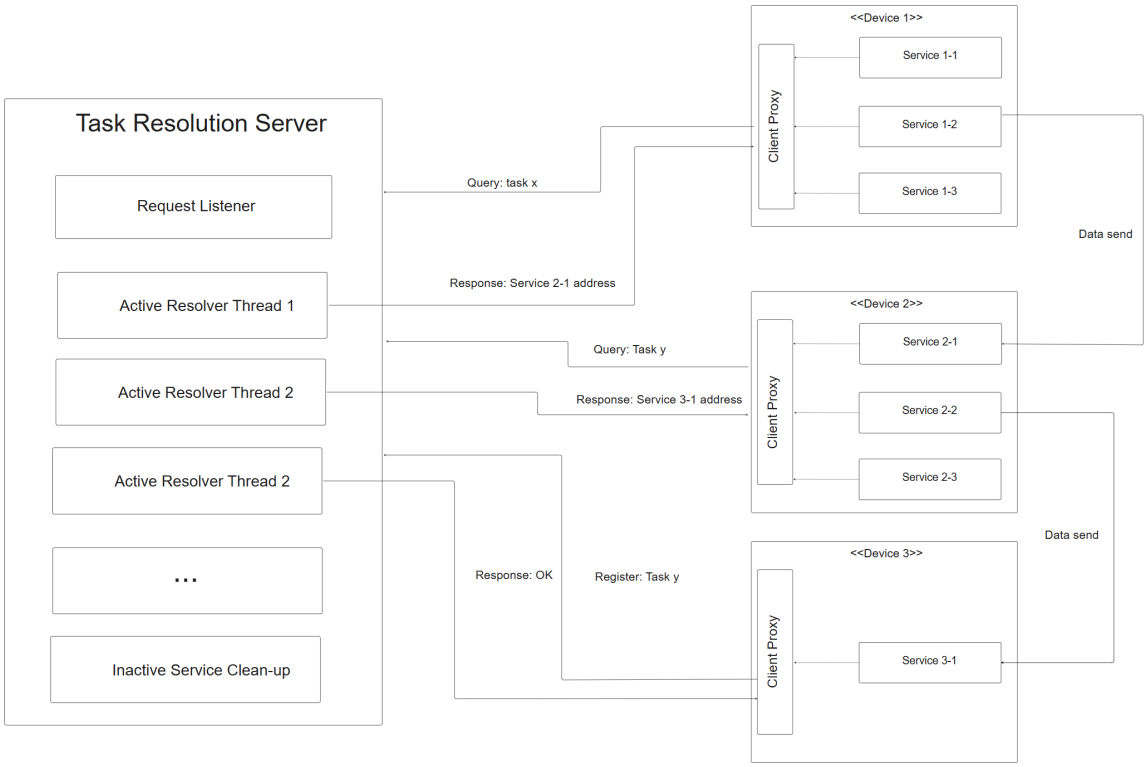

Figure 2: Client machines operating in server architecture

as a weighting. This ensures that services running on less powerful machines are not swamped by being routed the same volume of traffic as a more powerful variant. During the heartbeat message, a service advertises its current capacity to inform load balancing.

3.2.2 Firewall-mode

In many real use cases of this project, while all machines are able to query and register with the TDS, not all services are allowed to communicate with each other. This is enforced by networking rules that are created and managed by a central authority in the system. The purpose of firewall-mode is to cross-reference a query for a service with the list of ip-address:ports that the requestor can communicate with. This way the server will only return a service address that the client can use.

3.3 Peer-to-Peer mode

The diagram below gives an overview of the peer to peer architecture. It shows how tasks are registered and then stored on the correct node in the network. It also demonstrates service queries and then the separate sending of data outside of the architecture for the process. In contrast to the centralised mode, all functionality of this architecture runs from the client-proxy. The P2P network is formed from one "bootstrap" node. All subsequent nodes that join are given a pre-existing node in the network as a bootstrap. When a node joins, it hashses its ip address to receive their node ID. This ID is capped at 256 bits forming a ring network.

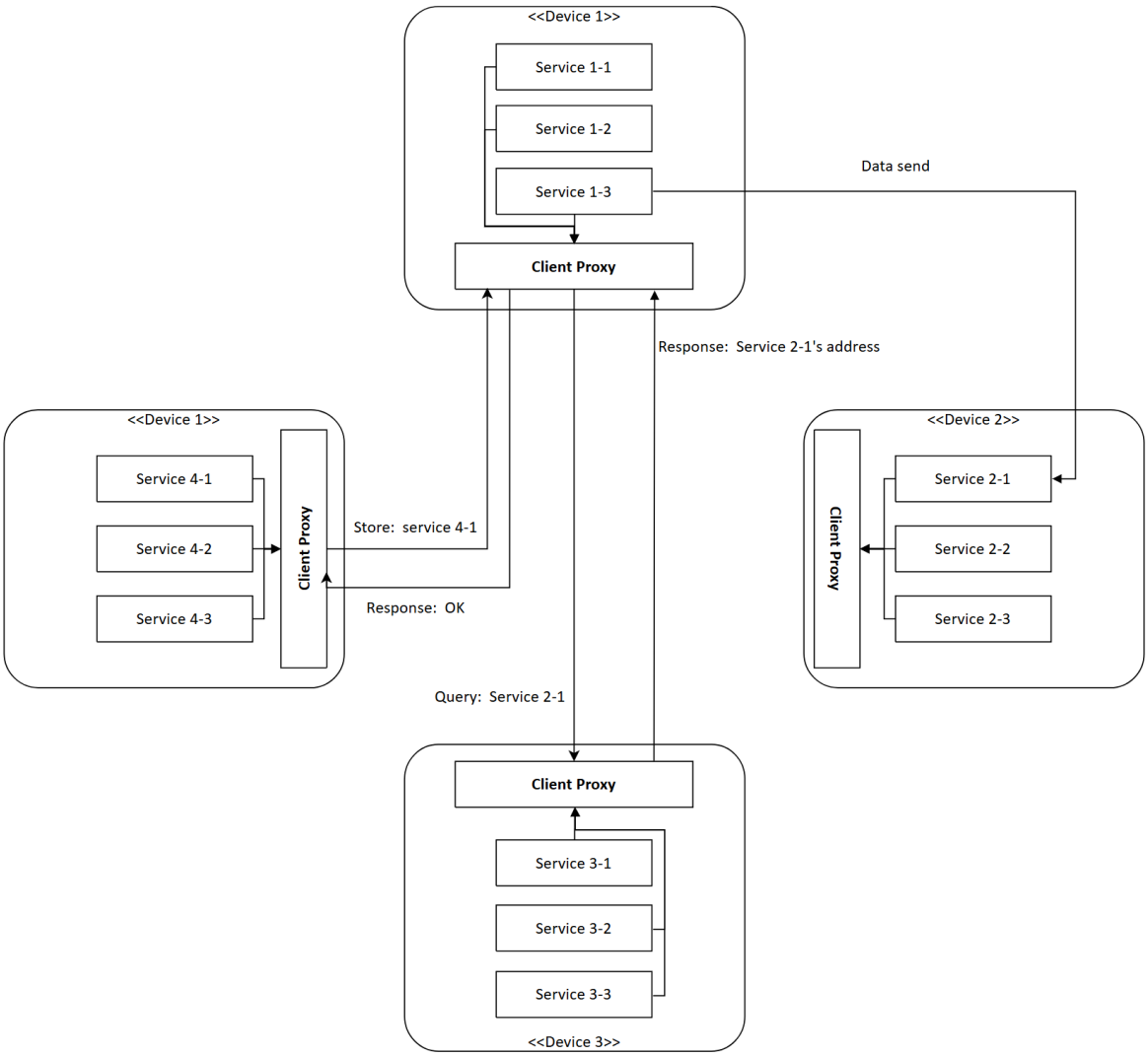

Figure 3: Client machines operating in P2P architecture

3.3.1 The Distributed Hash Table (DHT) and Service Discovery

The DHT is used to store the task:address mappings in this architecture. When a service registers a task with their client-proxy, the proxy hashes the service name. It then indexes its list of peers to find the node ID that is closest to this hash sending a STORE request to it with the service details. The storing node compares its peer list to the task hash to ensure it is the most appropriate candidate. If so, it stores the task in memory similarly to the cache in centralised mode.

A query follows the same path: the client-proxy hashes the query task and sends a query request to the node with the closest ID. If that node has that service stored it returns an address, if not then it returns NOTFOUND.

3.3.2 Eventual consistency and node discovery

The nature of the Distributed Hash Table makes it eventually consistent. Nodes are given an initial peer list from their bootstrap node. As more nodes join the network they discover them through store and query messages as, if they do not have the requesting node in their peer list, they add these nodes to their peer list. Additionally, to suppliment this, a perodic re-discovery process runs where nodes query their original bootstrap node for any peers that joined after them. This introduces extra computation onto bootstrap nodes but this can be balanced by carefully choosing more computationally powerful devices in the network to be bootstrap nodes.

3.3.3 K-Nearest Neighbours

On top of the previously discussed node discovery mechanisms, there are protections against the incorrect storage of tasks when naive nodes have not yet discoved all their peers. In the event that a node fails to select the correct node for a given task hash, usually because the correct node is not yet in their peer list, a node will reject a received store request if it does not believe itself to be the closest to the hash and tell the requestor the node they believe is responsible to proactively bridge the knowledge gap. This, however, does not eliminate the possibility of incorrectly storing a task as both nodes might not have the best candidate in their peer list. The implemented protection against this is to implement k-nearest-neighbours: the client-proxy finds the k closest nodes to their hash and sends a store request to all three. As well as providing protection against tasks not being stored correctly it allows for redundant storage of task mappings in the network in case of node loss. With this enabled, nodes will instead check if they are in the best k candidates for the hash instead.

4 Implementation details and testing

This section will first cover the challenges that were encountered while implementing the above design; it will then summarise functional and integration testing to give a holistic view of the implemented design.

4.1 Implementation Challenges

Here i will lay out key insights into areas of my project that presented key challenges to me. It details how I overcame them and, in particular, where I decided to cease further

work in the scope of the project timeframe.

4.1.1 Server Cache Performance

The implementation of the cache appeared simple but produced several challenges. Having completed the cache implementation, I ran perforance tests with a combination of cache hits and misses. Initially I found that cache misses were actually faster than hits! After increasingly deep investigation I found that, though queries should be read intensive, because they incremented query counts and round robin it was write locking significantly. My solution was two-fold: firstly, I batched query count updates to the database as they were not necessary until the next cache warming. I then researched in to Atomics for the round robin and with these implemented the cache became significantly faster than the database.

4.1.2 Peer-to-Peer Challenges

Having implemented and tested the P2P mode for this project, I found that it struggled to scale with large numbers of nodes. The issue pertained to the fact that, as the network grew, patterns of individual nodes communicating emerged. Many nodes were rarely contacted and therefore their routing tables were not updated with the latest information. In these cases the centralised approach was far more effective. What the network gained in robustness against node failure, it lost with scale.

4.1.3 Firewall-Mode

Firewall mode was a feature I implemented with the goal of allowing the TDS system (in centralised mode) to only serve up services that were routable for the requestor. This proved to be a significant challenge as the only method I devised was to import firewall rule file from the system administrator of any given application. This was deemed unacceptable as the separation of the TDS' routing table from the source of truth (the firewall) was a recipe for inconsistency. The feature was therefore not developed further but included as an artifact as further work could be done to improve the implementation.

5 Functional testing

perhaps write a test report

6 Realistic Application Testing

To demonstrate the practical utility of TDS, I implemented a mock-up of the system that inspired it. I created a highly abstracted version of a train ticketing, Oyster and contactless payment system.

The system specified can be found under the Simulation folder of the project repository.

It contains a set of programs that simulate a full distributed system with stations, payment gates, fare calculation and very rough PCTR-tokenisation to simulate cardholder environment structure. Key features include:

- Station computers to batch transactions
- Payment gates to process card and ticket presentations
- Fare calculation service to compute trip costs
- Multiple endpoints for ticket distribution to demonstrate load balancing

A full High-Level-Design document is available in the repository under `ticketing_system_High_Level_Design_Document.pdf`

All of these devices were interconnected by querying the TDS system for service discovery. Of key note was that, although many issues arose developing this system and setting up the environment, communication between devices was seamless. This was a testament to the robustness of the project as the only time I had to troubleshoot communication was when i forgot to start the TDS server itself.

7 Future Work

Several enhancements would improve TDS capabilities:

Replication: Add multi-master replication for centralized mode high availability. Raft consensus could provide strong consistency while tolerating failures.

Health checking: Implement active health probes instead of passive heartbeats. Server could periodically verify service reachability and automatically deregister failed instances.

Service metadata: Support arbitrary key-value tags for services enabling filtering queries (e.g., "version=2.0", "datacenter=us-east").

Multi-datacenter: Extend P2P mode with geographic awareness for locality-based routing.

Metrics export: Integrate with Prometheus for comprehensive monitoring and alerting.

REST API: Add HTTP REST interface alongside binary protocol for broader language support and web-based tooling.

7.1 Conclusions

References

[1] Stoica, I., Morris, R., Karger, D., Kaashoek, M. F., & Balakrishnan, H. (2001). *Chord: A scalable peer-to-peer lookup service for internet applications*. ACM SIGCOMM Computer Communication Review, 31(4), 149-160.

[2] HashiCorp. (2021). *Consul: Service Mesh for Microservices*. https://www.consul.io/

[3] CoreOS. (2020). *etcd: A distributed, reliable key-value store*. https://etcd.io/

[4] Hunt, P., Konar, M., Junqueira, F. P., & Reed, B. (2010). *ZooKeeper: Wait-free coordination for internet-scale systems*. USENIX Annual Technical Conference, 10(8), 9.

[5] Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., & Lewin, D. (1997). *Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the World Wide Web*. ACM Symposium on Theory of Computing, 654-663.

[6] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.

[7] Donovan, A. A., & Kernighan, B. W. (2015). *The Go Programming Language*. Addison-Wesley Professional.

[8] Cardellini, V., Colajanni, M., & Yu, P. S. (1999). *Dynamic load balancing on web-server systems*. IEEE Internet Computing, 3(3), 28-39.

[9] Ratnasamy, S., Francis, P., Handley, M., Karp, R., & Shenker, S. (2001). *A scalable content-addressable network*. ACM SIGCOMM Computer Communication Review, 31(4), 161-172.